EXPLORING 1.58-BIT QUANTIZATION IN NON-LLM CONTEXTS

by

Michael Tran

A Thesis
Submitted to the
Graduate Faculty
of
George Mason University
In Partial fulfillment of
The Requirements for the Degree
of
Master of Science
Computer Science

Committee:

Daniel Barbará — Dr. Daniel Barbará, Thesis Director

Dr. ThanhVu Nguyen, Committee Member

Sam M. Sainju — Mr. Shambhu Sainju, Committee Member

Dr. Songqing Chen, Department Chair

Date: August 14, 2026

Fall Semester 2026
George Mason University
Fairfax, VA

Exploring 1.58-Bit Quantization in Non-LLM Contexts

A thesis submitted in partial fulfillment of the requirements for the degree of Master of Science at George Mason University

By

Michael Tran
Bachelor of Science
George Mason University, 2023

Director: Dr. Daniel Barbará, Professor
Department of Computer Science

Fall Semester 2026
George Mason University
Fairfax, VA

Copyright © 2026 by Michael Tran
All Rights Reserved

Acknowledgments

I would like to express my deepest gratitude to my advisor, Dr. Daniel Barbará, for his invaluable guidance, patience, and expertise throughout this research. I would also like to thank my committee members, Dr. ThanhVu Nguyen and Professor Shambhu Sainju, for their time and dedication to reviewing this work. Without their support, this thesis would not have been possible.

Table of Contents

Page

List of Tables . . . vi
List of Figures . . . vii
Abstract . . . viii
1 Introduction . . . 1
2 Background . . . 3
2.1 Quantization Fundamentals . . . 3
2.2 CUDA Inference Kernels . . . 4
2.3 BitNet b1.58 . . . 5
3 Related Works . . . 10
4 GEMV Kernel Experiments . . . 12
4.1 Evaluation Metrics . . . 12
4.2 Kernel Compatibility . . . 13
4.3 Multilayer Perceptron . . . 14
4.3.1 Architecture . . . 14
4.3.2 Experimental Setup . . . 14
4.3.3 Results . . . 15
4.3.4 Discussion . . . 20
4.4 Residual Network . . . 21
4.4.1 Architecture . . . 21
4.4.2 Experimental Setup . . . 22
4.4.3 Results . . . 22
4.4.4 Discussion . . . 27
4.5 Vision Transformer . . . 28
4.5.1 Architecture . . . 29
4.5.2 Experimental Setup . . . 30
4.5.3 Results . . . 30
4.5.4 Discussion . . . 35
5 BitBLAS Experiments . . . 37
5.1 Architecture and Evaluation Metrics . . . 37

5.2 Experimental Setup 38
5.3 Multi-Layer Perceptron 38
5.3.1 Results 38
5.3.2 Discussion 40
5.4 Residual Network 41
5.4.1 Results 41
5.4.2 Discussion 42
5.5 Vision Transformer 42
5.5.1 Results 42
5.5.2 Discussion 44
6 Cross-Model Comparison 45
7 Limitations 47
8 Conclusion 49

List of Tables

Table		Page
4.1	MLP Architecture	15
4.2	MLP Training Performance	16
4.3	Total Inference Time, Test Accuracy, Batch Latency, and Throughput Across MLP Models	16
4.4	MLP Parameter Counts and Checkpoint Sizes	19
4.5	Modified ResNet-18 Architecture	22
4.6	ResNet Training Performance	23
4.7	Total Inference Time, Test Accuracy, Batch Latency, and Throughput Across ResNet Models	23
4.8	ResNet Parameter Counts and Checkpoint Sizes	27
4.9	ViT-Tiny Architecture	29
4.10	ViT Training Performance	30
4.11	ViT Inference Performance	31
4.12	ViT Parameter Counts and Checkpoint Sizes	34
5.1	BitBLAS MLP Training Performance	39
5.2	BitBLAS MLP Inference Performance	39
5.3	BitBLAS MLP Parameter Counts and Checkpoint Sizes	40
5.4	BitBLAS ResNet Training Performance	41
5.5	BitBLAS ResNet Inference Performance	42
5.6	BitBLAS ViT Training Performance	43
5.7	BitBLAS ViT Inference Performance	43

List of Figures

Figure		Page
2.1	A high level overview detailing the different kernels used in LLM decoding and classification inference.	6
2.2	A high level overview of the forward passes during training and inference.	7
4.1	Baseline MLP training and validation loss over epochs.	17
4.2	BitLinear MLP training and validation loss over epochs.	18
4.3	Baseline MLP training and validation accuracy over epochs.	18
4.4	BitLinear MLP training and validation accuracy over epochs.	19
4.5	Baseline ResNet training and validation loss over epochs.	24
4.6	BitLinear ResNet training and validation loss over epochs.	25
4.7	Baseline ResNet training and validation accuracy over epochs.	25
4.8	BitLinear ResNet training and validation accuracy over epochs.	26
4.9	Baseline ResNet training and validation loss over epochs.	32
4.10	BitLinear ResNet training and validation loss over epochs.	33
4.11	Baseline ResNet training and validation accuracy over epochs.	33
4.12	BitLinear ResNet training and validation accuracy over epochs.	34

Abstract

EXPLORING 1.58-BIT QUANTIZATION IN NON-LLM CONTEXTS

Michael Tran, MS

George Mason University, 2026

Thesis Director: Dr. Daniel Barbará

While 1.58-bit quantization has demonstrated remarkable success in scaling large language models, its applicability to other deep learning architectures remains largely unexplored. This thesis investigates the implementation of 1.58-bit quantization in three vision models: a multilayer perceptron, a convolutional neural network, and a vision transformer. We quantify the benefits of ternary quantization by evaluating accuracy preservation, checkpoint size reduction, and inference speedups. Furthermore, we compare low-bit inference between the standard cuBLAS library, BitNet's custom W2A8 kernel, and the BitBLAS GEMM kernels. Our results show that accuracy is preserved within 5% across all architectures, and that checkpoint sizes are reduced by up to 15.8x. No inference speedups were observed over the cuBLAS baselines due to small matrix sizes and kernel limitations, though we demonstrate that BitBLAS's GEMM kernels drastically outperform BitNet's custom GEMV kernel. We conclude that 1.58-bit quantization shows potential for checkpoint size reduction at minimal cost to model accuracy, and that low-bit kernels designed for LLM decoding do not generalize to vision model inference.

Chapter 1: Introduction

In recent years, the Transformer architecture has seen widespread adoption in deep learning models, culminating in powerful large language models (LLMs) such as ChatGPT and Gemini. While these LLMs demonstrate remarkable capabilities, they suffer from several limitations including high memory consumption and energy costs [1]. To address these shortcomings, Ma et al. introduced BitNet b1.58, a 1-bit transformer architecture that builds upon the earlier BitNet work by Wang et al. [1, 2]. The key innovation is BitLinear: a drop-in substitute for the `nn.Linear` layer. BitLinear constrains weights to the ternary values $\{-1, 0, 1\}$ while also quantizing activations to 8 bits [2]. Their method of quantization-aware training (QAT) results in smaller checkpoints, greater throughput, and minimal accuracy loss [2]. Despite its success in LLMs, BitLinear has not been thoroughly studied in non-LLM contexts.

This thesis addresses this gap by evaluating 1.58-bit quantization in vision architectures. While our investigation initially focused on a multilayer perceptron (MLP) and a convolutional neural network (CNN), these models did not yield the expected inference speedups. We thus expanded our scope to include a vision transformer (ViT), which features a high density of linear layers that makes the architecture suitable for BitLinear experiments. We evaluate accuracy preservation, checkpoint size reduction, and inference speed using both BitNet b1.58's custom W2A8 kernel and the BitBLAS library. Our primary objective is to determine whether the benefits of 1.58-bit quantization reported for LLMs generalize to vision model inference. To support reproducibility, our code is publicly available at `https://github.com/CynosureNTS/ms_thesis`.

This thesis is organized as follows. Chapter 2 provides background information on quantization fundamentals, CUDA inference kernels, and BitNet b1.58. Chapter 3

describes the training setups, architectures, and results of our experiments using BitNet's GEMV kernel. Chapter 4 repeats the experiments using the BitBLAS library to address the kernel limitations identified in Chapter 3. Chapter 5 presents a cross-model comparison and a discussion of the findings. Chapter 6 discusses the limitations of our approach, and Chapter 7 offers our conclusions and potential directions for future work.

Chapter 2: Background

This section provides the essential background information required to contextualize this thesis. Concepts are framed conceptually at a high level for broad accessibility, delving into the low-level technical details only when necessary.

2.1 Quantization Fundamentals

Quantization is a technique applied to neural networks to reduce the bitwidth of weights and/or activations, reducing the model's memory footprint, storage requirements, and computational costs [3]. In many models, weights are stored as 32-bit (FP32) or 16-bit (FP16/BF16) floating point numbers where the bitwidth (32/16) denotes the number of bits each weight occupies in memory. Quantization seeks to reduce this bitwidth to values such as 8, 4, or fewer bits, often using integer formats, binary formats, or custom representations [3]. However, reducing the bitwidth often introduces quantization error (the difference between the original and quantized values), creating a tradeoff between accuracy and memory efficiency [3].

There are two main approaches to quantization: quantization-aware training (QAT) and post-training quantization (PTQ) [3]. In PTQ, weights are quantized to the desired bitwidth after all training has finished. Although simple to implement, PTQ often results in significant accuracy loss during inference because the model was not optimized to function under reduced precision [3]. To minimize accuracy degradation, QAT integrates quantization into the training or fine-tuning process [3]. Weights and activations are stored in high precision during training, but the use of scaling factors and clamping functions causes the network to adapt to lower precision constraints throughout

training. This ensures high accuracy during inference when the weights and activations are fully quantized to their target bitwidths.

2.2 CUDA Inference Kernels

In the context of deep learning, kernels implement the core mathematical operations that are used during training and inference, such as matrix multiplications and activation functions. CUDA kernels are functions that execute on the GPU, launching many threads in parallel to perform computations efficiently [4]. Kernels operate on data using a tiling strategy where matrices are partitioned into equally sized, non-overlapping tiles [4]. Each tile is processed by a group of threads called a thread block, which perform arithmetic operations on the tile to produce a new tile [4]. The resulting output tiles are combined to form the output matrix [4]. Tile dimensions must be powers of 2 and are fixed at compile time [4]. If matrix dimensions are not evenly divisible by the compiled tile size, then out-of-bounds errors or incorrect results may occur [4]. The custom W2A8 (denoting 2-bit weights and 8-bit activations) kernel used throughout this thesis specifically requires that matrix dimensions adhere to the following constraints: the input features must be a multiple of 128 and the outfeatures must be a multiple of 32. This compile-time constraint is fundamental to understanding the dimension compatibility issues encountered in this thesis.

To manage the execution of these kernels, programmers utilize CUDA streams. CUDA streams are a queue-like abstraction that allow the programmer to sequence operations like kernel launches [4]. The operations of a stream are executed sequentially, though multiple streams may execute concurrently under specific circumstances [4].

General Matrix Multiplications (GEMMs) are defined as $C = \alpha AB + \beta C$ where A and B are matrices, α and β are scalars, and C is a pre-existing matrix which is overwritten by the output [5]. In the forward pass of a fully-connected layer, the weight matrix corresponds to A with dimensions (M, K), the incoming activations correspond to

B with dimensions (K, N), and α and β are typically one and zero [5]. In other words, M corresponds to batch size, K corresponds to the number of input features, and N corresponds to the number of output features. Generalized Matrix-Vector Multiplication (GEMV) operations are a special case of GEMM operations where either $M = 1$ or $N = 1$, computing the product of a matrix and a vector [5].

During autoregressive LLM inference, each decode step generates exactly one new token, requiring a GEMV ($M = 1$) operation. BitNet b1.58's custom W2A8 kernel optimizes low-bit inference by hardcoding the $M = 1$ constraint into the kernel logic. By contrast, classification models process entire batches of samples simultaneously, naturally lending themselves to GEMM operations with M equal to the batch size. Figure 2.1 summarizes the difference between LLM decoding and classification inference.

2.3 BitNet b1.58

In 2024, Ma et al. introduced a 1-bit LLM called BitNet b1.58 where every weight is ternary $\{-1, 0, 1\}$. BitNet b1.58 is based on the BitNet architecture, where `nn.Linear` layers of the transformer blocks are replaced by `BitLinear` layers. BitLinear has two different implementations between training and inference, so we designate the inference version as a `BitLinearKernel` layer. During training, `BitLinear` layers apply RMS normalization, weight quantization, and activation quantization before performing the standard `F.linear` operation. At inference, `BitLinearKernel` layers replace the `F.linear` operations with a custom `bitnet_int8xint2_linear` function which calls the low-bit kernel. BitNet b1.58 undergoes QAT with high-precision weights, but uses 8-bit activations and 2-bit weights during inference. Ternary weights have three possible states which require $\log_2(3) \approx 1.58$ bits to represent, giving the model its name.

The weights are quantized to -1, 0, or 1 using the absmean quantization function, in which the weight matrix is scaled by its average absolute value before rounding each value to the nearest integer among $\{-1, 0, 1\}$ [2]. During forward passes, the activations

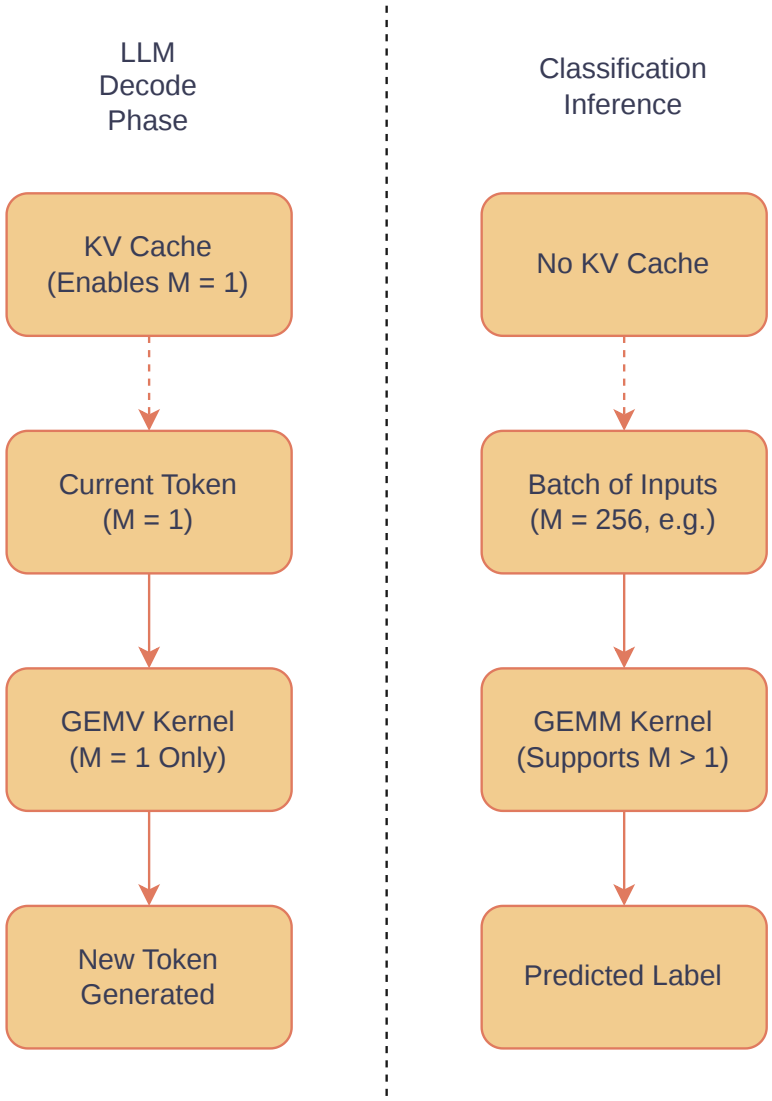


Figure 2.1: A high level overview detailing the different kernels used in LLM decoding and classification inference.

are quantized using a modified absmax quantization function [2]. After the matrix multiply operations, the activations are cast back to high precision before the next layer. This cast between layers is necessary because normalization functions and activation functions (e.g. ReLU and GeLU) require high precision. A summary of the forward passes is depicted in Figure 2.2.

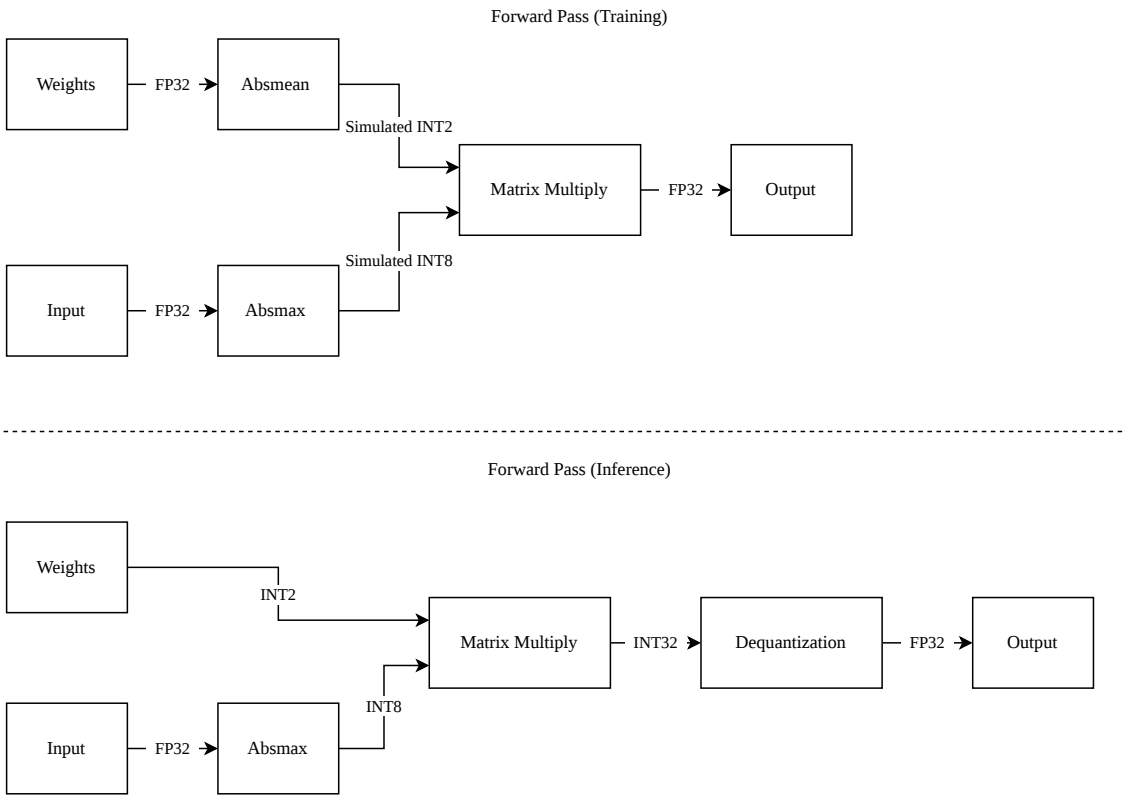


Figure 2.2: A high level overview of the forward passes during training and inference.

To achieve efficient GPU inference, BitNet b1.58 uses a custom GEMV CUDA kernel designed for W2A8 matrix operations [6]. During offline quantization, groups of 4 ternary weights are packed into a single 8-bit integer (INT8) for storage [6]. During inference, the kernel loads the packed weights from the GPU high bandwidth memory

(HBM) into the GPU's faster on-chip shared memory (SRAM) where it unpacks the values before performing the matrix multiplication with the 8-bit activations [6]. Finally, it dequantizes the result back to high precision.

Since the weights of BitNet b1.58 are $\{-1, 0, 1\}$, the matrix multiplication operations which dominate computation reduce to the more energy-efficient addition and subtraction operations [1]. For example, $A = W_1x_1 + W_2x_2$ where $W_1 = 1$ and $W_2 = -1$ becomes $A = (1)x_1 + (-1)x_2$ which reduces to a subtraction operation. Therefore, the energy consumption of the matrix multiply operations in BitNet b1.58 is significantly lower than that of vanilla transformers which rely heavily on the less energy-efficient matrix multiply operations [1, 2]. For power-constrained hardware, the energy savings of BitNet also directly translate to faster computation within the same energy budget [2].

Compared to similar-sized FP16 models, BitNet b1.58 performed similarly in terms of perplexity and end-task performance while being more cost-effective in terms of latency, memory, throughput, and energy consumption [2]. Ma et al. found that starting from a 3B parameter model size, BitNet b1.58 matched an FP16 LLaMA LLM in terms of perplexity, while being 2.71 times faster and using 3.55 times less GPU memory [2]. They found that BitNet b1.58 70B is 4.1x faster than the LLaMA LLM baseline due to the time cost of `nn.Linear` growing with model size [2]. Ma et al. similarly found that as model size increases, BitNet b1.58 becomes increasingly more energy efficient compared to full precision baselines [2]. This is because the majority of BitNet b1.58's computations are INT8 additions, while LLaMA baselines used both FP16 addition and FP16 multiplication [2]. As a result, they estimate that BitNet b1.58 reduces arithmetic operations energy consumption by a factor of 71.4 [2].

Furthermore, the process of transferring model weights from DRAM to the memory of the on-chip accelerator can significantly slow down inference [2]. Because 1.58-bit LLMs have a lower memory footprint in terms of both capacity and bandwidth compared to full-precision models, they are quicker and more cost-effective to load from DRAM, resulting in faster inference [2]. At the 70B parameter scale evaluated across two

80GB A100 GPUs, BitNet b1.58 supported up to 11 times the batch size of the baseline LLaMA LLM while achieving an 8.9 times greater throughput [2].

Chapter 3: Related Works

Since its introduction, BitNet b1.58 has inspired further exploration of 1.58-bit quantization outside of LLMs. Yuan et al. [7] applied 1.58-bit quantization to ViTs, while Nielsen et al. [8, 9] evaluated its efficacy in small language models (SLMs), CNNs, MLPs, and graph neural networks (GNNs).

Yuan et al. explore 1.58-bit quantization in a ViT-L model, comparing classification performance across 16-bit, 8-bit, 4-bit, and 1.58-bit precision [7]. On CIFAR-10, their 1.58-bit ViT incurs only a 1.67% accuracy drop compared to the 16-bit baseline, while outperforming the 8-bit and 4-bit variants. They report a similar pattern when evaluating their ViTs on the ImageNet-1k dataset.

Nielsen et al. evaluate 1.58-bit quantization in SLMs and in CNNs at varying scales [8]. They introduce a modified Absmean quantization function that uses the median instead of the mean, and also perform ablation studies to identify the effects of learning rate and weight decay on 1.58-bit model accuracy. Their 1.58-bit SLMs achieve near state-of-the-art results, while their CNNs surpass state-of-the-art baselines. However, they find that their modified Absmean function only situationally improves model accuracy. In a subsequent study, Nielsen et al. [9] expand their evaluation to MLPs, GNNs, and several language model architectures (encoder-only, encoder-decoder, and decoder-only). Across tasks including the XOR problem, text classification, graph node classification, and masked language modeling, their 1.58-bit models maintain competitive performance compared to full-precision baselines, with the exception of the decoder-only model.

The works of Yuan et al. and Nielsen et al. show that 1.58-bit models achieve competitive performance in non-LLM contexts, but these works are primarily concerned with model accuracy. Yuan et al. briefly mention that their 1.58-bit ViT achieves a

greater throughput than the 16-bit counterpart, but inspection of their public code repository reveals that they are not using BitNet b1.58's optimized low-bit kernel. The code repository for Nielsen et al.'s SLM and CNN experiments reveals that they are also not using BitNet b1.58's kernel, and their second paper does not provide a code repository at all. This thesis differentiates itself from these prior works by more thoroughly evaluating the inference speedups of 1.58-bit quantization, and by using the exact GEMV kernel of the original BitNet b1.58 architecture.

Chapter 4: GEMV Kernel Experiments

In this chapter, we conduct several experiments to examine whether the advantages of 1.58-bit quantization persist in non-LLM contexts. In particular, this thesis focuses on accuracy preservation, memory savings, and inference speedups. To do this, we compare several baseline models with their quantized counterparts. The Baseline FP32 models use standard `nn.Linear` layers throughout. BitLinear models replace the `nn.Linear` layers with `BitLinear` layers which perform simulated ternary quantization during training and inference. After training, the weights of the BitLinear models are offline quantized for use with the Inference Kernel models. Inference Kernel models further replace `BitLinear` layers with `BitLinearKernel` layers and perform inference using the low-bit weights. Inference Kernel models are only used to evaluate inference performance using the 2-bit weights generated from QAT, so no separate training phase is conducted. All three model variants share identical training setups and hyperparameters, differing only in the linear layer implementation and weight storage format. We perform all experiments using a 40GB NVIDIA A100 in Google Colab. Where possible, we follow the architectures and training setups of prior literature to ensure that best practices are followed and to facilitate comparison with established results.

4.1 Evaluation Metrics

To understand the impact of QAT on model training, several key metrics are examined. First, we measure the difference in training time between Baseline and BitLinear models to quantify the computational overhead introduced by QAT operations. Next, validation accuracies after training are compared to evaluate any accuracy degradation caused by the replacement of `nn.Linear` layers with `BitLinear` layers. We additionally analyze training loss and accuracy curves over epochs to assess convergence behavior and training stability.

To verify the theoretical benefits of quantization, we measure model size, inference speed, latency, throughput, and test accuracy. Model size is characterized by both the on-disk checkpoint size and total parameter count. The checkpoint size includes weights, biases, and other remaining full-precision parameters. Total inference time, latency (measured as inference time per batch) and throughput (measured as samples processed per second) are measured using wall-clock time and GPU synchronization. Test accuracy measures the percentage of correctly classified samples on the held-out test set, and is useful for quantifying the accuracy degradation incurred by performing inference using low-bit weights.

We also perform three trials for each experiment (both training and inference) to gather averages and standard deviations of applicable metrics. We only discuss the standard deviations when they are anomalously high or otherwise noteworthy, such as when variance may affect the reliability of comparisons. We set the random seed equal to the run number to ensure reproducibility, and GPU warmup is applied for 5 batches to ensure an accurate measurement of the inference speed.

4.2 Kernel Compatibility

As described in Section 2.2, BitNet's custom GEMV kernel has strict tiling requirements. To meet these requirements, we pad certain layers of each architecture to ensure that the resulting matrices' K dimension is divisible by 128 and N dimension is divisible by 16. In particular, the MLP's input layer is padded from 784 to 800[1] neurons, and the output layer is padded to 32 neurons. For the CNN, the input to the fully-connected output layer is padded from 64 to 128 neurons, and the output is padded to 32 neurons. The ViT's embedding dimension of $K = 192$ is not evenly divisible by 128, so we attempted to pad it to 256 neurons. This introduced illegal memory access errors in the kernel, so we report our findings with the original unpadded $K = 192$. We apply the padding to both the Baseline and BitLinear models to ensure a fair comparison.

[1]See Section 7 for further discussion.

Additionally, we compile the custom W2A8 kernel with the dimensions of the expected matrix multiplications. Crucially, the kernel's internal logic is hardcoded for GEMV ($M = 1$) operations whereas our batch sizes are 512 and 128 ($M = 512$ and 128). To compensate, we modify the `bitnet_int8xint2_linear` function (which launches the kernel) to launch a CUDA stream for each sample in the batch. The use of CUDA streams allows the kernels to potentially execute in parallel. Each stream processes one row of the input and writes results to a preallocated output tensor. We call `torch.cuda.synchronize()` to wait for all streams to complete before returning the full batch output. Consequently, this approach requires calling the kernel up to 512 times instead of once, which introduces a significant amount of kernel launch overhead that may impact inference speed.

4.3 Multilayer Perceptron

We first test `BitLinear` layers in a multilayer perceptron. MLPs are an appropriate candidate for BitLinear experiments because the majority of their parameters are contained in `nn.Linear` layers, all of which can be replaced with `BitLinear` layers.

4.3.1 Architecture

We follow the architecture described in Zuo's work [10]. This architecture consists of the input layer, three fully connected hidden layers of sizes 512, 256, and 128, and the output layer. Each hidden layer is followed by ReLU and a dropout layer with dropout probability p = 0.25. The architecture is detailed in Table 4.1.

4.3.2 Experimental Setup

We test the MLPs on the classic MNIST dataset, which is composed of 70,000 images of 28×28 grayscale handwritten digits. The dataset is split into 50,000 training samples, 10,000 validation samples, and 10,000 test samples.

Table 4.1: MLP Architecture

Layer	Type	Output Size
1	Flatten	784
2	ConstantPad1d (784 → 800)	800
3	BitLinear	512
4	ReLU	512
5	Dropout (p=0.25)	512
6	BitLinear	256
7	ReLU	256
8	Dropout (p=0.25)	256
9	BitLinear	128
10	ReLU	128
11	Dropout (p=0.25)	128
12	BitLinear (sliced to 10)	32

Following the experimental setup of Zuo [10], we initialize the weights for each layer to follow a Gaussian distribution with zero mean and 0.01 standard deviation. Biases are initialized with zeros. We use the Adam optimizer and the cross-entropy loss function. Baseline and BitLinear models are trained for 50 epochs with a batch size of 512 and learning rate of 0.01. Once training is complete, we perform offline quantization of the BitLinear model's weights for use with the Inference Kernel model. We follow the absmean quantization implementation provided by Ma et al. [11].

4.3.3 Results

To verify the correctness of our model implementation, we compare our Baseline model to the results found by Zuo [10]. Our measurements differ minimally from the reference architecture, showing a discrepancy of only 0.23% in validation accuracy and 0.11% in test accuracy. These discrepancies are possibly attributed to the random weight initialization or the random batch shuffling during training.

Table 4.2: MLP Training Performance

Model	**Validation Accuracy (%)**	**Training Time (seconds)**
Baseline	98.14 ± 0.09	646.48 ± 0.90
BitLinear	97.94 ± 0.02	670.43 ± 2.36

We evaluate the training performance of the Baseline and BitLinear models over 50 epochs, with the results compiled in Table 4.2. The Baseline model achieves an average validation accuracy of 98.14% and completes training in 646.48 seconds. By comparison, the BitLinear model reaches an average validation accuracy of 97.94% (a 0.2% decrease) and finishes training in 670.43 seconds, representing an approximate 3.7% reduction in training speed.

Table 4.3: Total Inference Time, Test Accuracy, Batch Latency, and Throughput Across MLP Models

Model	**Inference Time (s)**	**Slowdown vs Baseline**	**Test Acc. (%)**	**Latency (ms)**	**Throughput (samples/s)**
Baseline	2.12 ± 0.01	$1.00\times$	98.22 ± 0.02	106.21 ± 0.33	4707.69 ± 14.62
BitLinear	2.19 ± 0.01	$1.03\times$	98.07 ± 0.02	109.48 ± 0.40	4567.08 ± 16.75
Inference Kernel	3.60 ± 0.01	$1.70\times$	98.07 ± 0.02	179.83 ± 0.74	2780.46 ± 11.51

Next, we compare inference between the Baseline, BitLinear, and Inference Kernel models, with the results summarized in Table 4.3. The BitLinear model exhibits an approximate 3.3% performance reduction across inference time, latency, and throughput metrics relative to the Baseline. Meanwhile, the Inference Kernel model exhibits an approximate 70% increase in inference time and latency compared to the Baseline, with 41% reduction in throughput. All three models achieve similar test accuracy, performing within 0.15 percentage points of each other. The BitLinear and Inference Kernel models achieved the exact same average test accuracy, though they varied on a per-run basis due to a difference in the order of dequantization operations.

We also evaluate training stability and convergence behavior by analyzing the plots of training loss (Figures 4.1 and 4.2) and accuracy (Figures 4.3 and 4.4) over epochs. In the Baseline model, the training loss drops rapidly in the first 10 epochs and continues decreasing gradually toward 0 by epoch 50, while the validation loss begins increasing around epoch 10. This widening gap suggests overfitting on the training data. The BitLinear model displays similar behavior, but the validation loss plateaus without the upward trend seen in the Baseline. This suggests that the BitLinear model is more resilient to overfitting, possibly due to a regularization effect caused by the simulated quantization. The BitLinear model's loss values start higher than those of the Baseline model, though both models have roughly the same loss values by the end of training. This suggests that the BitLinear model initially has a harder time training due to the additional quantization constraints, but quickly matches Baseline performance.

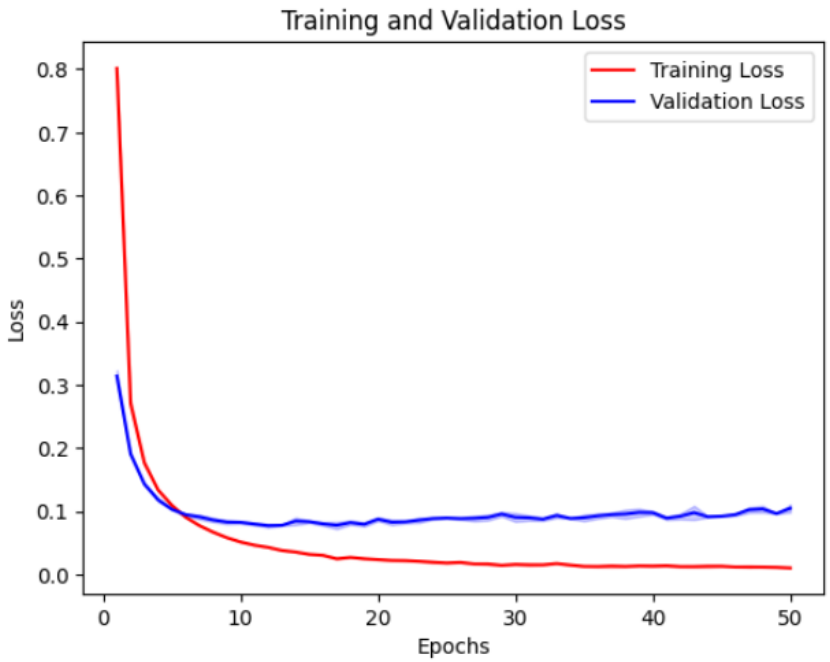


Figure 4.1: Baseline MLP training and validation loss over epochs.

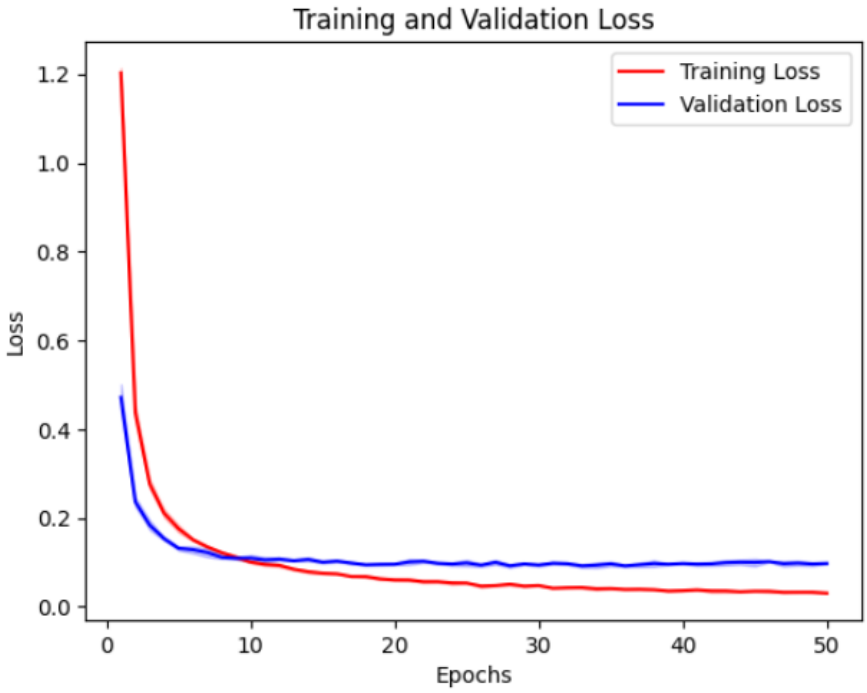


Figure 4.2: BitLinear MLP training and validation loss over epochs.

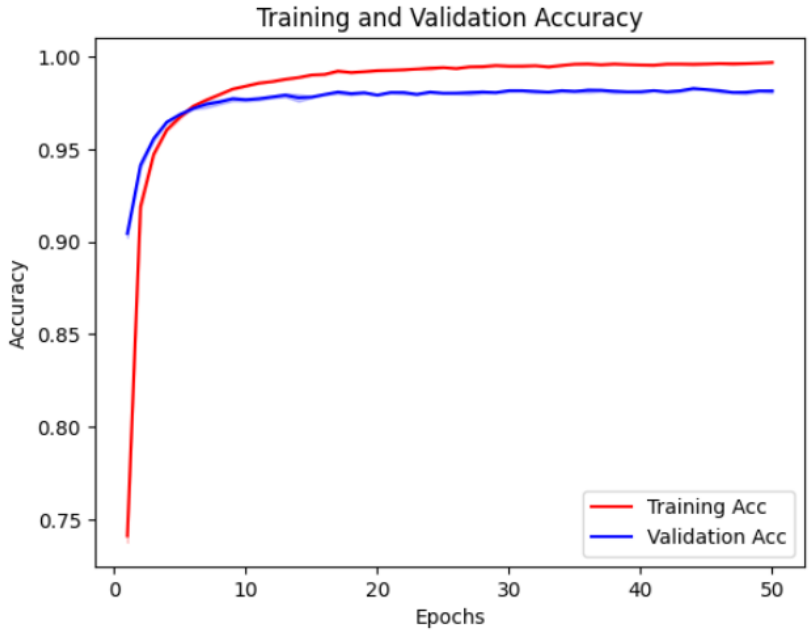


Figure 4.3: Baseline MLP training and validation accuracy over epochs.

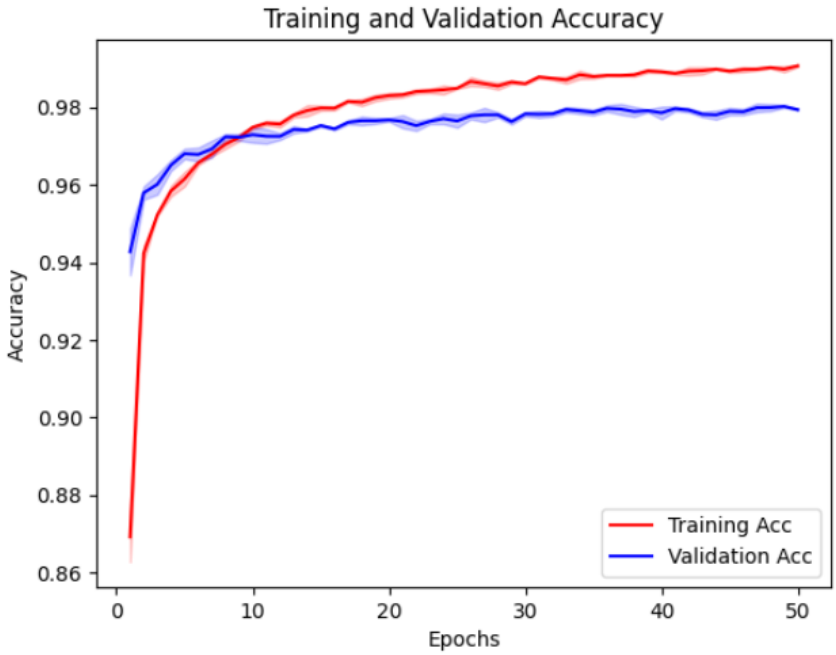


Figure 4.4: BitLinear MLP training and validation accuracy over epochs.

Despite differing y-axis scales on their respective plots, both the Baseline and BitLinear models converge to similar final accuracies. Additionally, the accuracy curves of the BitLinear model exhibit greater variance than those of the Baseline model, which is likely due to the QAT operations within the BitLinear layers. Whereas the Baseline model is free to make small, continuous adjustments to its weights, the BitLinear model quantizes its weights to discrete ternary values. Small gradient updates may be insufficient to change a weight's ternary value, while larger updates can cause weights to snap discretely between values. This leads to the fluctuations seen in the BitLinear model's accuracy plot.

Table 4.4: MLP Parameter Counts and Checkpoint Sizes

Model	**Total Parameters**	**Trainable Parameters**	**Checkpoint Size (MB)**
Baseline	577,536	577,536	2.21
BitLinear	577,536	577,536	2.21
Inference Kernel	144,400	0	0.14

Finally, we compare parameter counts and checkpoint sizes between models, summarized in Table 4.4. The Baseline and BitLinear models have the exact same parameter counts and checkpoint sizes, which is expected since only the layer type changes and QAT simulates quantization in full precision. The Inference Kernel model reports approximately one quarter of the total parameters because four 2-bit weights are packed into each byte during offline quantization. This results in the 15.8x checkpoint size reduction that closely matches the theoretical 16x ratio between FP32 and INT2 storage. Additionally, the Inference Kernel model reports zero trainable parameters, reflecting that the weights have been frozen for inference.

4.3.4 Discussion

The BitLinear model's validation accuracy differs from the Baseline by only 0.2%, demonstrating that QAT successfully enables the model to adapt to the low-precision weights. The BitLinear model on average finished 50 epochs of training 3.7% slower than the Baseline model, which is likely attributed to the additional QAT operations that are present in the `BitLinear` layer. All three models achieve similar average test accuracy within 0.15 percentage points, suggesting that 1.58-bit quantization does not adversely affect model accuracy and supporting the accuracy claims made by Ma et al. [2].

The results also show that the BitLinear model is approximately 3.3% slower than the Baseline model in terms of inference time, throughput, and latency. This mirrors the 3.7% difference in training speed between these models, suggesting that the cause of the slowdown also stems from the additional RMSNorm, absmean, and absmax functions that the `BitLinear` layers have over the standard `nn.Linear` layers.

Compared to the Baseline model, the Inference Kernel model exhibits a 70% increase in inference time and latency, with a 41% reduction in throughput. Based on the findings of Ma et al., 1.58-bit quantization is expected to improve these metrics over the Baseline, especially as model size increases [2]. However, the observed decline in performance is likely caused by the necessary kernel changes described in Section 4.3.2.

Namely, we circumvent the custom GEMV kernel's $M = 1$ constraint by launching 512 CUDA streams, one for each sample in the batch. This equates to 2,048 kernel launches per batch (512 per layer $\times$ 4 layers) in the Inference Kernel model compared to just four kernel launches (1 per layer $\times$ 4 layers) per batch for the Baseline. This means that the Inference Kernel model performs 512 times more kernel launches per layer than the Baseline, introducing significant kernel launch overhead. A GEMM kernel supporting $M > 1$ would reduce kernel launches from 2,048 to just 4 per batch, potentially enabling the inference speedups reported by Ma et al.

Despite the inference speed slowdown, the Inference Kernel model's checkpoint size is 15.8x smaller than the Baseline. Because the MLP consists entirely of BitLinear layers, all the weights are quantized to INT2, nearly achieving the theoretical 16x compression ratio from FP32 to INT2 storage.

4.4 Residual Network

We next implement `BitLinear` layers in a CNN. CNNs are appropriate candidates for BitLinear experiments because they typically have fully-connected layers after the convolutional layers that are prime replacement targets[2].

We specifically use a modified ResNet-18 based on the architecture described by He et al. [12]. ResNets are well-benchmarked architectures with established performance on the CIFAR-10 dataset, making them a good baseline for comparison. Note that only the fully-connected output layer is replaced; convolutional layers do not use `nn.Linear` layers and are therefore not suitable for replacement.

4.4.1 Architecture

For our CNN experiments, we use a modified ResNet-18 architecture based on the CIFAR-10 experiments of He et al. [12]. The architecture is summarized in Table 4.5.

[2]See Section 4.4.4 for further discussion.

Table 4.5: Modified ResNet-18 Architecture

Layer	Type	Output Channels
1	Conv2d 3×3, stride=1, padding=1, no bias	16
2	BatchNorm2d	16
3	ReLU	16
4	BasicBlock ×2 (3×3 conv, stride=1)	16
5	BasicBlock ×2 (3×3 conv, stride=2)	32
6	BasicBlock ×2 (3×3 conv, stride=2)	64
7	AdaptiveAvgPool2d (output 1×1)	64
8	ConstantPad1d (64 → 128)	128
9	BitLinear (in=128, out=32, sliced to 10)	32

4.4.2 Experimental Setup

We test our custom ResNet-18 models against the CIFAR-10 dataset, which consists of 60,000 32×32 color images distributed across 10 classes. These images are split into training, validation, and test sets using a 40,000:10,000:10,000 ratio. Following the experimental setup of He et al. [12], we train the models with a batch size of 128 while applying data augmentation that consist of random crop and random horizontal flip with the per-pixel mean subtracted.

We initialize the weights using Kaiming normal initialization and train each model for 182 epochs. We use cross-entropy loss and stochastic gradient descent (SGD) with momentum of 0.9 and weight decay of 0.0001. The learning rate is initially set to 0.1 and is divided by 10 after 32,000 and 48,000 mini-batch iterations.

4.4.3 Results

First, we compare our Baseline model's test accuracy to the results of He et al. [12]. Our ResNet-18-based model does not exactly match their configurations, but we compare it against their most comparable model. Their 20-layer model with 0.27 million parameters achieves a test accuracy of 91.25%, while our Baseline model with 0.178 million parameters achieves a test accuracy of 89.91%.

The BitLinear model trained approximately 8% slower than the Baseline and achieved a validation accuracy within 0.08 percentage points, consistent with the minimal accuracy degradation observed in the MLP experiments. Training results are summarized in Table 4.6.

Table 4.6: ResNet Training Performance

Model	**Training Time (s)**	**Validation Accuracy (%)**
Baseline	851.38 ± 5.47	0.8991 ± 0.0017
BitLinear	920.98 ± 2.41	0.8983 ± 0.0007

At inference, all three models achieve nearly identical test accuracy, differing by only 0.02 percentage points. In terms of inference speed, the BitLinear model is approximately 3.5% slower than the Baseline across all metrics, while the Inference Kernel model has approximately 76% higher inference time and latency with 43% lower throughput. Table 4.7 summarizes these results.

Table 4.7: Total Inference Time, Test Accuracy, Batch Latency, and Throughput Across ResNet Models

Model	**Inference Time (s)**	**Slowdown vs Baseline**	**Test Acc. (%)**	**Latency (ms)**	**Throughput (samples/s)**
Baseline	0.58 ± 0.01	$1.00\times$	89.91 ± 0.29	7.30 ± 0.19	17350.42 ± 441.28
BitLinear	0.60 ± 0.02	$1.03\times$	89.89 ± 0.03	7.57 ± 0.21	16744.48 ± 465.65
Inference Kernel	1.02 ± 0.04	$1.76\times$	89.89 ± 0.03	12.89 ± 0.51	9832.34 ± 383.38

Next, we analyze the convergence behavior of Baseline and BitLinear models during training. The accuracy and loss plots exhibit sharp shifts at around 100 and 153 epochs, as seen in Figures 4.5 through 4.8. At these points, the learning rate scheduler reduces the learning rate by a factor of 10. The learning rate starts relatively high at 0.1, and these subsequent reductions facilitate finer-grained optimization that stabilizes

training. Both models show a growing gap between training and validation loss after epoch 100 which suggests mild overfitting, though validation accuracy remains stable throughout.

Similar to the results of the MLP experiments, the BitLinear model experiences a more jagged loss curve with greater variance across runs. This is likely due to BitLinear's discrete weight updates discussed in Section 4.3.3. The final performance of both models is nearly identical, again suggesting that replacing `nn.Linear` layers with `BitLinear` layers does not significantly alter model performance during training.

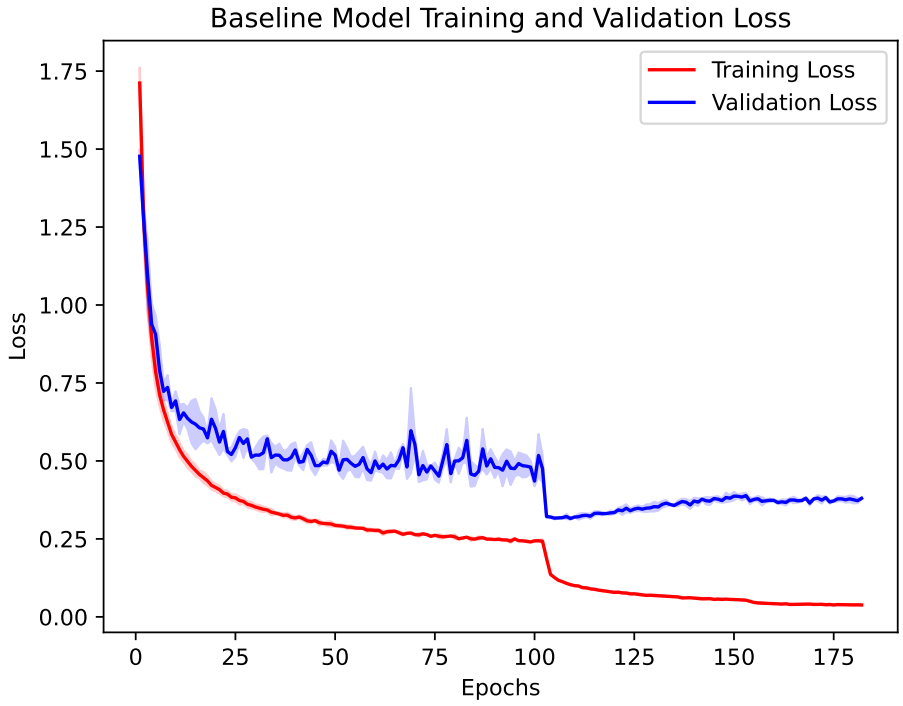


Figure 4.5: Baseline ResNet training and validation loss over epochs.

Finally, we compare the parameter counts and checkpoint sizes between the Baseline and quantized models, summarized in Table 4.8. As with the MLP experiments, the Inference Kernel model displays fewer total parameters than the Baseline and BitLinear models because four 2-bit quantized weights are packed into a single byte.

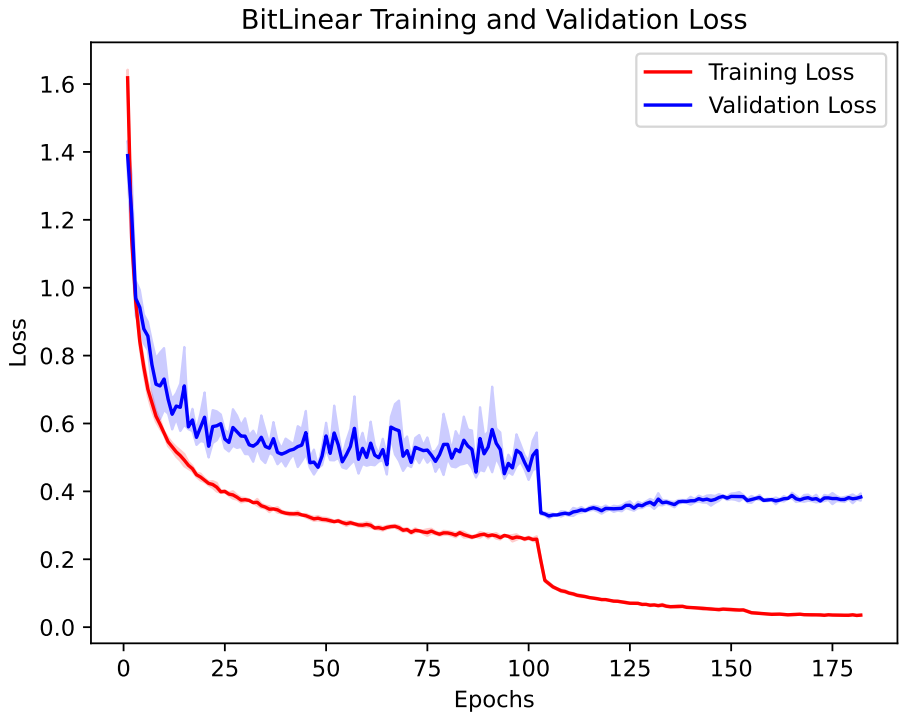


Figure 4.6: BitLinear ResNet training and validation loss over epochs.

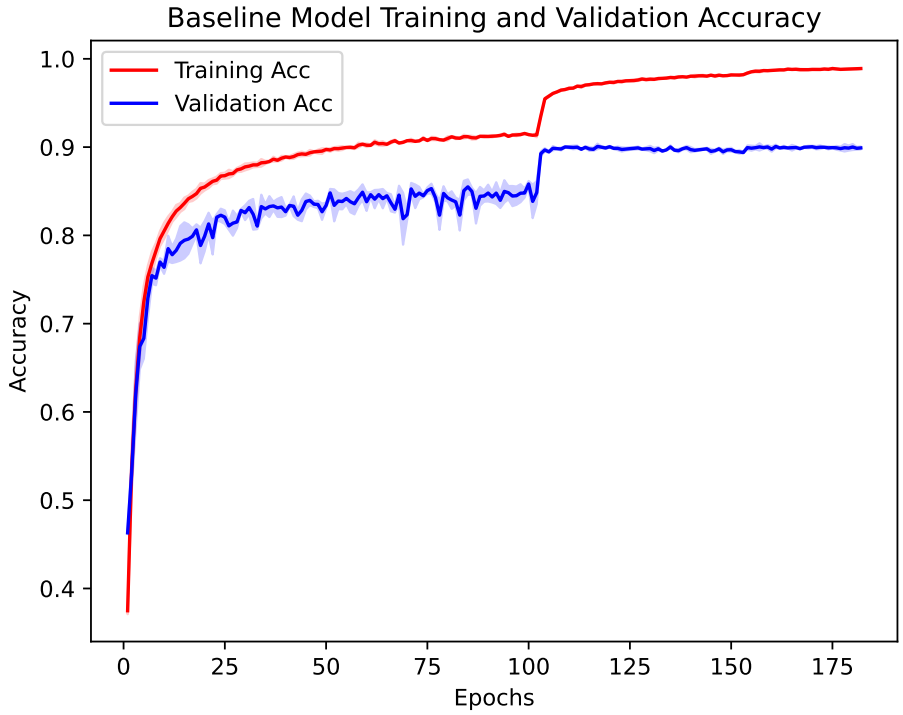


Figure 4.7: Baseline ResNet training and validation accuracy over epochs.

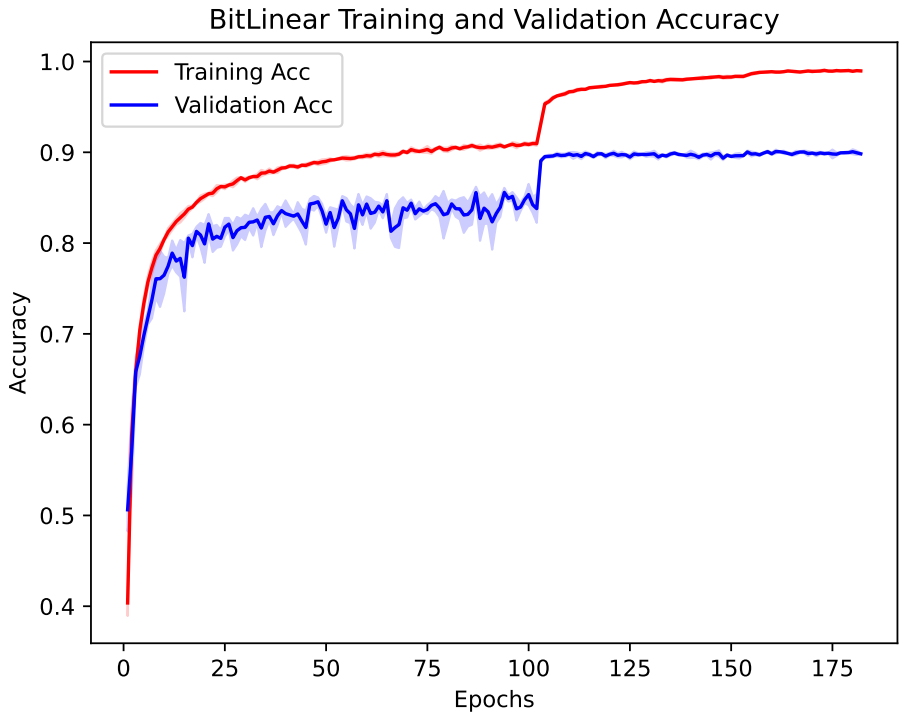


Figure 4.8: BitLinear ResNet training and validation accuracy over epochs.

However, unlike in the MLP experiments, the parameter count of the Inference Kernel model is not roughly one fourth that of the Baseline. This is because the MLP models were entirely fully-connected, where all the weights were quantized to low bitwidths. By contrast, our ResNet models only replace a single layer with a `BitLinear` layer, meaning that the majority of the weights are left unquantized. The ResNet Inference Kernel model's weights are dominated by those of the convolution layers, resulting in only a minor reduction in checkpoint size relative to the Baseline. Additionally, the ResNet Inference Kernel model reports 144,608 trainable parameters, whereas the Inference Kernel model of the MLP experiments reported zero. This is because the weights of the convolutional layers were not explicitly frozen. However, this has no practical impact because The Inference Kernel model is only used for inference, where no optimizer is applied and no weight updates take place.

Table 4.8: ResNet Parameter Counts and Checkpoint Sizes

Model	**Total Parameters**	**Trainable Parameters**	**Checkpoint Size (MB)**
Baseline	178,704	178,704	0.72
BitLinear	178,704	178,704	0.72
Inference Kernel	175,636	144,608	0.70

4.4.4 Discussion

As in the MLP experiments, 1.58-bit quantization preserved test accuracy. The quantized models differed from the Baseline by only 0.02%, an even smaller degradation than the 0.15% observed in the MLP experiments. This is possibly because the ResNet has fewer BitLinear layers than the MLP, meaning fewer opportunities for quantization errors to accumulate.

The BitLinear model trained 8% slower than the Baseline, whereas the slowdown was 3.7% in the MLP experiments. This is an unexpected result because the ResNets only have one `BitLinear` layer compared to the four of the MLP, meaning the MLPs should theoretically experience more QAT overhead than the ResNets. The additional slowdown experienced by the ResNet could stem from differences in the training setup, such as the difference in optimizer or the use of a learning rate scheduler for the ResNet.

The percentage difference in inference performance between Baseline and quantized ResNet models (76% increased inference time and latency, 43% less throughput) appears to mirror the percentage differences observed in the MLP models (70% and 40%). Given that the ResNets are dominated by convolution layers instead of linear layers unlike MLPs, this is a counter-intuitive result. However, this phenomenon can be explained by calculating the amount of overhead per kernel launch. For MLPs, the difference in total inference time between the Baseline and Inference Kernel models is $3.60s - 2.12s = 1.48s$. This is the total overhead incurred by kernel launches during inference. If we divide 1.48 seconds by the total number of kernel launches (40,000, one launch per sample times 4 layers), then the overhead per kernel launch is approximately 0.037 milliseconds.

Performing the same calculations for the ResNets ($1.02 - 0.58 = 0.44$ seconds divided by 10,000 calls) results in a consistent per-kernel-launch overhead of 0.044 ms. Framed this way, the mirrored percentage differences are mostly coincidental. Note that the overhead of these calculations also includes the cost of activation quantization and dequantization operations.

As with the MLPs, the custom CUDA kernel is not compatible with the ResNet's dimensions out of the box, forcing our model to launch one kernel call per sample per BitLinear layer. With a fully compatible GEMM kernel, our ResNet could theoretically perform inference in $10,000/128 \approx 78$ total kernel calls. Instead, our Inference Kernel model makes 10,000 kernel calls during inference, resulting in approximately 128 times more kernel launches than a GEMM kernel would require. This negates the theoretical inference time speedups of 1.58-bit quantization.

Based on checkpoint sizes, ResNets are not ideal candidates for 1.58-bit quantization. The Baseline checkpoint had an on-disk size of 0.72 MB, whereas the quantized checkpoint had an on-disk size of 0.70 MB. This result differs greatly from the theoretical 16x reduction that arises from quantizing from FP32 to INT2. The primary reason our ResNets don't strongly benefit from 1.58-bit quantization is that the majority of their layers are convolutional and thus unsuitable for replacement with `BitLinear` layers. In fact, only the single fully-connected output layer of our ResNet was suitable for replacement. Because the majority of the ResNet's weights remain unquantized, we do not observe the substantial reduction in checkpoint size achieved in the MLP experiments, where all parameters were quantized to 2 bits.

4.5 Vision Transformer

To evaluate whether the inference speedups of 1.58-bit quantization scale with model size, we apply our testing methodology to a ViT. Ma et al. report a 2.71x speedup and a 4.1x speedup over FP16 LLaMA baselines at the 3B and 70B model sizes, so we use a ViT architecture containing 5.5 million parameters to investigate whether the speedups emerge

at that scale [2]. For comparison, this ViT has 10 times the number of parameters of the MLP, and 30 times the number of parameters of the ResNet. Furthermore, the ViT architecture is a suitable candidate for BitLinear experiments because its structure more closely matches that of BitNet b1.58, and because it has a large number of linear layers suitable for `BitLinear` layer replacement.

4.5.1 Architecture

Table 4.9: ViT-Tiny Architecture

Layer	**Type**	**Output Shape**
1	Patch Embedding (16×16, Conv2d)	196 × 192
2	Class Token + Positional Embedding	197 × 192
3–14	Transformer Block ×12	197 × 192
	LayerNorm	197 × 192
	QKV Projection (BitLinear)	197 × 576
	Multi-Head Self-Attention (3 heads)	197 × 192
	Output Projection (BitLinear)	197 × 192
	LayerNorm	197 × 192
	MLP fc1 (BitLinear)	197 × 768
	GELU	197 × 768
	MLP fc2 (BitLinear)	197 × 192
15	LayerNorm	197 × 192
16	Class Token Extraction	192
17	Classification Head (BitLinear)	112

We use the ViT-Tiny architecture which adopts the ViT design of Dosovitskiy et al. [13]. ViT-Tiny is pretrained on ImageNet-21k and fine-tuned on ImageNet-1K according to the training setup of Steiner et al. [14]. The architecture is summarized in Table 4.9.

4.5.2 Experimental Setup

We test the ViT-Tiny (referred to as simply the ViT henceforth) against the ImageNet-100 dataset. This balanced dataset consists of 224×224 images of 100 classes. The training set consists of 1250 images per class, and the validation and test sets consist of 50 images for each class. The training images are augmented with random resized crop, random horizontal flip, RandAugment with two operations, color jitter, and random erasing at p = 0.25.

We train the model from scratch for 15 epochs. We use cross-entropy loss, AdamW optimizer, a learning rate of 1e-4, weight decay of 0.01, and a batch size of 128. As with the MLP and ResNet experiments, we compile the kernel with the expected matrix multiply dimensions and modify the kernel launch function to launch a CUDA stream per sample in a batch, circumventing the $M = 1$ kernel constraint.

4.5.3 Results

Table 4.10: ViT Training Performance

Model	**Training Time (s)**	**Validation Accuracy (%)**
Baseline	3320.01 ± 7.39	37.99 ± 0.47
BitLinear	3368.24 ± 9.45	32.97 ± 0.32

The BitLinear model completed training in 3368.24 seconds, which is 1.45% slower than the Baseline. It also achieved a validation accuracy of 32.97%, 5.02 percentage points below the Baseline. This difference in accuracy is much higher than in the MLP and ResNet experiments, where validation accuracy differed by 0.2 and 0.08 percent, respectively. This difference is likely due to accumulating quantization error over the ViT's 49 `BitLinear` layers. By contrast, the MLP had 4 `BitLinear` layers, and the

ResNet had only one. See Section 7 for further discussion. A summary of the ViT training results is presented in Table 4.10.

A summary of the Inference results can be found in Table 4.11. The BitLinear model achieves an average test accuracy of 37.28%, which is again roughly 5 percentage points below the Baseline. The Inference Kernel model achieved a test accuracy of only 6.98%, which is barely above random guessing in a 100-class dataset. This poor accuracy is likely attributed to an incompatible tiling strategy, as described in Section 2.2. Specifically, the ViT's embedding dimension $K = 192$ is not evenly divisible by the kernel's internal tile size of 128. This causes the kernel to process only the first 128 of 192 input features, resulting in incorrect outputs.

The BitLinear model performed inference approximately 1% slower than the Baseline across all inference speed metrics. However, the Inference Kernel model was 335 times slower than the Baseline in terms of total inference time and latency, and 99.7% slower in terms of throughput. This extreme slowdown is due to the kernel's $M = 1$ constraint, as mentioned in previous sections. With a batch size of 128 and 197 tokens per image, each forward pass requires $128 \times 197 = 25,216$ kernel calls per `BitLinearKernel` layer compared to just 1 per layer for the Baseline. With 49 total `BitLinearKernel` layers and approximately 39 batches, the total number of W2A8 kernel calls during inference is 48,187,776. From the ResNet experiments, we calculated the approximate overhead per kernel launch to be 0.044 ms. The estimated total kernel launch overhead is therefore approximately 2,120 seconds, consuming roughly 93% of the total inference time.

Table 4.11: ViT Inference Performance

Model	**Inference Time (s)**	**Slowdown vs Baseline**	**Test Acc. (%)**	**Latency (ms)**	**Throughput (samples/s)**
Baseline	6.77 ± 1.27	1.00×	42.34 ± 0.23	169.31 ± 31.80	762.05 ± 126.51
BitLinear	6.84 ± 1.14	1.01×	37.28 ± 0.39	170.99 ± 28.49	749.83 ± 113.01
Inference Kernel	2269.35 ± 5.67	335.21×	6.98 ± 0.18	56733.65 ± 141.84	2.20 ± 0.01

Training accuracy and loss curves for the Baseline and BitLinear models can be found in Figures 4.9, 4.10, 4.11, and 4.12. Notably, both models achieve greater validation accuracy than training accuracy. This is because the training samples undergo aggressive data augmentation, whereas the validation and test samples do not. Validation is therefore easier, leading to higher validation accuracy. Additionally, neither model has converged by the end of training, as evidenced by the accuracy and loss curves still improving at epoch 15. Similar to the MLP and ResNet experiments, the BitLinear model's accuracy and loss curves appear to have more variance than the Baseline, likely due to the effects of ternary quantization explained in Section 4.3.3.



Figure 4.9: Baseline ResNet training and validation loss over epochs.

The number of parameters and checkpoint sizes of the models can be found in Table 4.12. The minor difference in parameter counts between the Baseline and BitLinear models is due to a minor architectural difference. This is because the BitLinear layers



Figure 4.10: BitLinear ResNet training and validation loss over epochs.



Figure 4.11: Baseline ResNet training and validation accuracy over epochs.

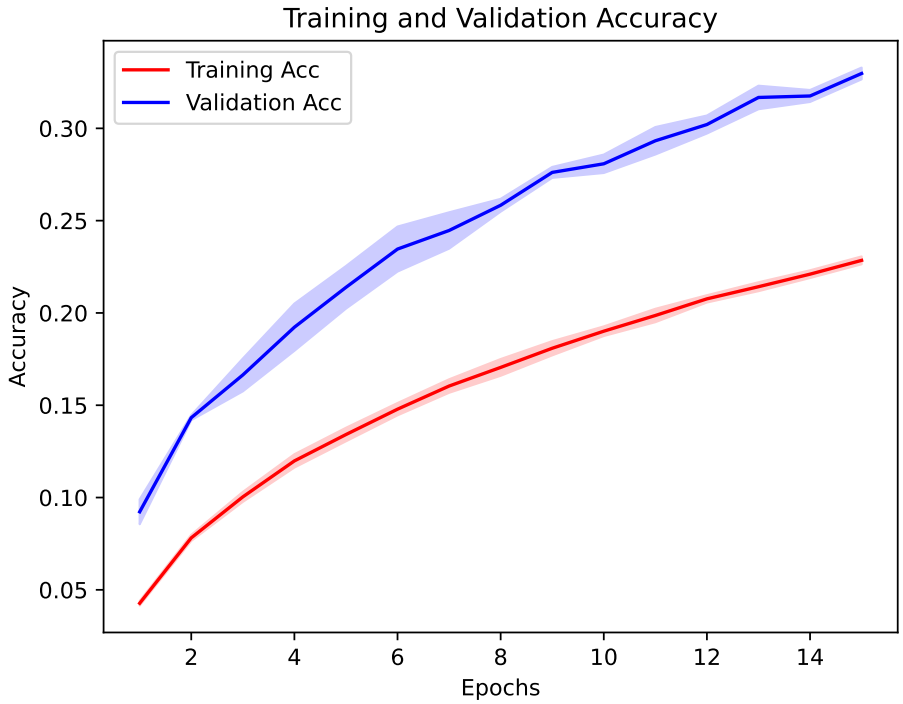


Figure 4.12: BitLinear ResNet training and validation accuracy over epochs.

Table 4.12: ViT Parameter Counts and Checkpoint Sizes

Model	**Total Params**	**Trainable Params**	**Checkpoint Size (MB)**
Baseline	5,546,032	5,546,032	21.21
BitLinear	5,525,296	5,525,296	21.12
Inference Kernel	1,527,940	195,264	2.07

were created with `bias=False`, whereas the original linear layers use `bias=True` by default. This results in the Baseline model having 0.37% more parameters than the BitLinear model, and a 0.09 MB larger checkpoint.

The Inference Kernel model has 27.6% of the total number of parameters as that of the Baseline model, as four 2-bit weights are packed into each byte. Despite this, the Inference Kernel's checkpoint size is only 10.2x smaller than the Baseline, instead of the theoretical 16x. This is because the ViT has several unquantized components such as positional and patch embeddings that are stored in FP32.

4.5.4 Discussion

The Inference Kernel model achieved a test accuracy of only 6.98%, likely due to tile compatibility issues as explained in the preceding section. This drastically reduced accuracy demonstrates the impact of a mismatch between the custom W2A8 kernel's tiling dimensions and the dimensions of the model architecture. It also highlights the necessity of padding certain layer dimensions to ensure proper tiling.

The ViT had the least training slowdown percentage of any tested architecture at 1.45%. This is because the ViT's attention mechanisms dominate computation, so any slowdowns caused by the `BitLinear` layer's extra QAT operations are less impactful. This suggests that in transformer-based architectures, the slowdown caused by this implementation of QAT can be negligible.

The Inference Kernel model is 335x slower than the Baseline with 99.7% reduced throughput. As explained in Section 4.5.3, this tremendous slowdown is caused by our attempt to circumvent the $M = 1$ kernel constraint by spawning multiple CUDA streams instead. The result is 48 million kernel calls over the course of inference which dominate inference time. This slowdown demonstrates the importance of using a compatible kernel, and it also demonstrates that our method of overcoming the $M = 1$ constraint is not a viable solution in larger models.

The Inference Kernel model's checkpoint is 10.2x smaller than the Baseline, a smaller reduction than the 15.8x observed in the MLP experiments. This difference arises because the ViT architecture contains unquantized components that are incompatible with BitLinear layer replacement, including positional embeddings, patch embeddings, and LayerNorm weights. This outcome is consistent with the 3.55x reduction reported by Ma et al., whose LLM architectures also contain large unquantized embedding tables [2]. Together, these results indicate that memory savings from 1.58-bit quantization scale directly with the proportion of BitLinear layers in the architecture.

Chapter 5: BitBLAS Experiments

One of the theoretical benefits of 1.58-bit quantization is an improvement in inference speed. However, our experiments with Ma et al.'s [2] custom W2A8 GEMV kernel have not attained this result. We suspect that the lack of inference time speedup can be attributed to the kernel's hardcoded GEMV ($M = 1$) matrix operations, which were designed specifically for LLM decoding with the BitNet-b1.58 2B-4T architecture. In practice, using this kernel for classification tasks significantly slows inference due to a dramatic increase in kernel launches stemming from our modified kernel launch function. Therefore, we attempt to demonstrate the inference time speedups of 1.58-bit quantization by instead using a different set of kernels from the BitBLAS library.

Microsoft's BitBLAS library was specifically designed to support mixed-precision BLAS operations on GPUs, like the W2A8 operations central to this thesis [15]. Crucially, the library contains both GEMV and the more general GEMM kernels [15]. Although the BitBLAS repository has been archived, it has been tested on BitNet-b1.58, reporting an 8x speedup for GEMV operations and a 2x speedup for GEMM operations over the standard cuBLAS library [15]. By replacing the custom GEMV kernel with BitBLAS's GEMM kernels, we hope to achieve inference time speedups even outside LLM contexts. We replace the custom GEMV kernel with the BitBLAS kernels in all of our architectures (MLP, ResNet, ViT) and report our findings below.

5.1 Architecture and Evaluation Metrics

Because we are performing the same experiments as in Chapter 3 but with the BitBLAS kernels, we use the exact same evaluation metrics. In particular, we are interested in test accuracy, total inference time, latency, and throughput. The architectures between GEMV

and BitBLAS experiments are also identical, but with minor adjustments to padding in some layers for BitBLAS tiling compatibility. Furthermore, The swap from the custom W2A8 kernel to BitBLAS kernels only affects inference when these kernels are called. The hyperparameter setups are exactly the same, so the resulting loss and accuracy curves are nearly identical. We omit these redundant plots for brevity, though they may be viewed in their respective notebook files in within our GitHub repository[1].

5.2 Experimental Setup

When testing the BitBLAS kernels, we use the same datasets and follow the same experimental setups as used in the custom GEMV kernel experiments. Although the BitBLAS kernels accommodate values of M greater than 1, we are still required to compile the kernel with matrix dimensions that can be tiled evenly. We use the compilation scripts in the BitBLAS GitHub repository [15] to compile operators for our specific matrix dimensions and verify correctness via reference checks. These reference checks verify correctness by comparing kernel outputs against standard `torch.matmul` reference results. As in previous experiments with the custom GEMV kernel, we apply padding to the layers of our architectures to the nearest BitBLAS-compatible dimension where necessary to meet kernel dimensionality requirements.

5.3 Multi-Layer Perceptron

5.3.1 Results

After switching to the BitBLAS kernels, validation and test accuracies are well preserved in both training and inference. Validation accuracy dropped by 0.15% between Baseline and BitLinear models, and test accuracy dropped by 0.18% between Baseline and Inference Kernel models. This result almost perfectly matches the MLP experiments with

[1] `https://github.com/CynosureNTS/ms_thesis`

the custom W2A8 kernel, further demonstrating that QAT and 1.58-bit quantization do not significantly affect model accuracy. The MLP training results are found in Table 5.1.

The Inference Kernel model exhibited a 4.4% increase in total inference time and latency relative to the Baseline, with a 4.1% reduced throughput. While these metrics fall short of the expected speedups of low-bit quantization, this performance gap is likely attributable to the models' small matrix dimensions. At this scale, the operations fail to fully stress GPU memory bandwidth, preventing the theoretical speedup from materializing. A summary of the inference performance results can be found in Table 5.2.

Compared to the custom W2A8 kernel MLP experiments, the BitBLAS MLP models have roughly 2.8% more parameters and 2.7% larger checkpoints. This is due to a minor difference in the amount of padding applied to the input layers. Since the kernels use different tiling strategies, the padding required to conform to each kernel differs slightly. A summary of the total parameters and checkpoint sizes can be found in Table 5.3.

Table 5.1: BitBLAS MLP Training Performance

Model	**Training Time (s)**	**Validation Accuracy (%)**
Baseline	624.10 ± 2.07	98.14 ± 0.03
BitLinear	649.09 ± 1.13	97.99 ± 0.02

Table 5.2: BitBLAS MLP Inference Performance

Model	**Inference Time (s)**	**Slowdown vs Baseline**	**Test Acc. (%)**	**Latency (ms)**	**Throughput (samples/s)**
Baseline	2.04 ± 0.00	$1.00\times$	98.26 ± 0.06	101.87 ± 0.24	4908.47 ± 11.73
BitLinear	2.10 ± 0.03	$1.03\times$	98.08 ± 0.16	104.84 ± 1.33	4769.82 ± 60.94
Inference Kernel	2.13 ± 0.05	$1.04\times$	98.08 ± 0.16	106.39 ± 2.43	4702.06 ± 105.66

Table 5.3: BitBLAS MLP Parameter Counts and Checkpoint Sizes

Model	**Total Params**	**Trainable Params**	**Size (MB)**
Baseline	593,920	593,920	2.27
BitLinear	593,920	593,920	2.27
Inference Kernel	148,484	0	0.15

5.3.2 Discussion

As discussed in Section 2.3, the inference speedups reported by Ma et al. stem from reduced HBM to SRAM transfer times enabled by smaller model weights. At the scale of our MLP, all weight matrices easily fit within the memory of the on-chip accelerator, resulting in no improvements to inference speed. This suggests that 1.58-bit quantization is more effective in larger models as reported by Ma et al., which will be examined further by applying the BitBLAS kernels on the larger ViT architecture [2].

Although the MLP results did not demonstrate the speedups expected from low-bit quantization, there was a significant inference speed improvement between the custom GEMV kernel and the BitBLAS GEMM kernels. In the GEMV MLP experiments, the Inference Kernel model performed inference 70% slower with 41% reduced throughput. By switching to the BitBLAS kernels, inference only slowed by 4.4% with a 4.1% reduced throughput. This highlights the fact that the custom GEMV kernel was designed specifically for LLM decoding. By switching to a compatible GEMM kernel, our models were able to reduce the number of kernel calls from 2,048 (512 per layer $\times$ 4 layers) per batch to 4 (1 per layer $\times$ 4 layers) per batch, significantly reducing the kernel launch overhead.

5.4 Residual Network

5.4.1 Results

The BitBLAS ResNet results largely mirror those of the GEMV experiments. The BitLinear model trained 7.7% slower than the Baseline, consistent with the 8% slowdown observed in the GEMV experiments, and achieved a validation accuracy within 0.15 percentage points of the Baseline. Table 5.4 summarizes the training results.

The Baseline model completes inference in 0.70 ± 0.22 seconds, has a latency of 8.85 ± 2.75 ms, and a throughput of $15{,}527.27 \pm 3{,}956.09$ samples per second. These metrics have a coefficient of variation of roughly 30%, which are anomalously high and unlike the inference results of the BitLinear and Inference Kernel models. The Baseline inference results should be evaluated with caution and warrant additional experiments. The inference results are summarized in Table 5.5.

As with the GEMV kernel experiments, no inference speedup was observed over the Baseline since the ResNet has only one small `BitLinear` layer whose weight matrix fits easily within the GPU's on-chip accelerator. However, the Inference Kernel model demonstrated a substantial performance improvement under BitBLAS than in the earlier GEMV experiments. Total inference time was comparable to the Baseline rather than 76% slower as previously observed with the custom GEMV kernel.

Table 5.4: BitBLAS ResNet Training Performance

Model	**Training Time (s)**	**Validation Accuracy (%)**
Baseline	830.76 ± 0.98	90.02 ± 0.25
BitLinear	894.60 ± 0.48	89.81 ± 0.11

Table 5.5: BitBLAS ResNet Inference Performance

Model	**Inference Time (s)**	**Test Acc. (%)**	**Latency (ms)**	**Throughput (samples/s)**
Baseline	0.70 ± 0.22	89.91 ± 0.03	8.85 ± 2.73	15527.27 ± 3956.09
BitLinear	0.59 ± 0.02	90.24 ± 0.20	7.42 ± 0.22	17072.91 ± 493.78
Inference Kernel	0.61 ± 0.03	90.23 ± 0.19	7.78 ± 0.33	16293.57 ± 667.87

5.4.2 Discussion

The BitBLAS experiments on the ResNet again suggest that 1.58-bit quantization does not significantly lower model accuracy. Low-bit inference using the BitBLAS kernels significantly outperforms the GEMV kernel, though it does not perform inference faster than the cuBLAS Baseline. This is partially because our model only has one `BitLinear` layer, so any performance speedups gained through quantization will be overshadowed by the convolution operations that dominate inference computation. Given that CNNs generally only have a few layers suitable for `BitLinear` layer replacement, most of the weights will not be quantized to low-bit. This makes CNNs poor candidates for 1.58-bit quantization, matching our findings from Section 4.4.4.

5.5 Vision Transformer

5.5.1 Results

The BitBLAS ViT training results are nearly identical to those of the GEMV experiments. Training times differ by less than 0.5% and validation accuracies are within 0.4 percentage points. This confirms that switching from the GEMV kernel to the BitBLAS kernels has no effect on training. Training results are summarized in Table 5.6.

Table 5.6: BitBLAS ViT Training Performance

Model	**Training Time (s)**	**Validation Accuracy (%)**
Baseline	3304.36 ± 2.62	37.99 ± 0.31
BitLinear	3343.55 ± 3.40	32.57 ± 0.14

The BitLinear model achieves a test accuracy of 36.97%, a 4.56 percentage point reduction compared to the Baseline. This is consistent with the 5.06 percentage point reduction observed in the GEMV ViT experiments. This gap is possibly due to quantization error accumulation across 49 layers, compounded by the absence of RMSNorm in our `BitLinear` layers. This design choice was carried over from the GEMV ViT experiments and is further discussed in Section 7. Additionally, both the Baseline and BitLinear inference metrics exhibited elevated variance, mirroring the execution anomalies observed in the BitBLAS ResNet experiments. This fluctuation is possibly caused by the GPU resource sharing of the Google Colab environment.

Table 5.7: BitBLAS ViT Inference Performance

Model	**Inference Time (s)**	**Test Acc. (%)**	**Latency (ms)**	**Throughput (samples/s)**
Baseline	6.86 ± 1.59	41.53 ± 0.24	171.48 ± 39.76	764.29 ± 152.49
BitLinear	6.54 ± 1.00	36.97 ± 0.65	163.39 ± 25.12	781.70 ± 108.40
Inference Kernel	26.81 ± 0.35	36.91 ± 0.39	670.24 ± 8.75	186.53 ± 2.42

The Inference Kernel model achieves a test accuracy of 36.91%,which is comparable to the BitLinear model. The Inference Kernel completed inference 3.9x slower than the Baseline, but is a major improvement over the 335x slowdown observed in the GEMV experiments. This 3.9x slowdown is due to kernel tiling constraints. For our ViT, M = batch size $\times$ sequence length = $128 \times 197 = 25{,}216$. Even though the GEMM kernel supports the compilation of M values greater than 1, it does not support every M. We

instead compile the kernel with $M = 256$ and break the input into 99 chunks. We therefore make 99 kernel launches per layer per batch, which equates to a total of 99 chunks $\times$ 49 layers $\times$ 40 batches = 194,040 kernel launches. This is a major improvement over the 48 million kernel launches required by the GEMV experiments, but is still 99 times more launches than the 1,960 of the cuBLAS default.

5.5.2 Discussion

The BitBLAS ViT experiments were intended to demonstrate that 1.58-bit quantization provides greater inference speedups at larger model sizes as reported by Ma et al. [2]. Instead, we encountered kernel tiling constraints that prevented a demonstration of this speedup. For our ViT, M = batch size $\times$ sequence length = $128 \times 197 = 25{,}216$, which was too large for the BitBLAS library to compile directly.

We instead compiled the kernel for $M = 256$ and processed the input in chunks. While this approach is significantly faster than our GEMV solution thanks to BitBLAS's support for larger M, our approach is not a viable long-term solution. A proper GEMM kernel with support for large M values is needed to demonstrate the inference speedups expected at larger model sizes. Low-bit mixed-precision kernels are still a fledgling area of research, and do not yet provide the flexibility of the cuBLAS library which has seen decades of development. The lack of speedups is therefore a limitation of the current low-bit kernels, not of 1.58-bit quantization itself.

Chapter 6: Cross-Model Comparison

In this chapter, we compare the results of the GEMV and BitBLAS kernel experiments across all three architectures to identify patterns and draw conclusions about the applicability of 1.58-bit quantization to several non-LLM models.

Across all three architectures and both kernels, 1.58-bit quantization preserved model accuracy. The MLP and ResNet showed minimal test accuracy degradation of less than 0.18%[1], while the ViT showed a larger degradation at roughly 5%[2]. This suggests that accuracy degradation scales with larger model sizes, though this claim runs counter to the results reported by Ma et al. [2]. The larger accuracy degradation of the ViT may stem from an accumulation of quantization error across a larger number of `BitLinear` layers, or it may stem from our omission of the RMSNorm operation during ViT testing. See further discussion in Section 7.

All three architectures showed modest training slowdowns from the additional QAT operations, ranging from 1.45% for the ViT to 8% for the ResNet. Notably, the training slowdowns do not correlate with the number of `BitLinear` layers. The ResNet trained 8% slower despite having only 1 `BitLinear` layer, while the ViT trained only 1.45% slower despite having 49. The ResNet experiments suggest that optimizer choice or learning rates have an influence on training slowdowns, while the ViT experiments suggest that attention-heavy architectures overshadow QAT overhead.

Ma et al.'s [2] custom W2A8 GEMV kernel demonstrated limited compatibility across all architectures. The kernel tiled correctly for the MLP and ResNet after padding certain layers, but produced incorrect results for the ViT due to its $K = 192$ not being

[1]The BitBLAS ResNet inference results had anomalously high variance, so are omitted from this comparison.

[2]This 5% was observed between the Baseline and BitLinear models. The Inference Kernel model had an incompatible kernel, resulting in poor accuracy.

evenly divisible by the kernel's internal tile size of 128. Furthermore, the kernel's $M = 1$ constraint required launching the kernel once per sample per layer, resulting in catastrophic inference speed slowdowns as the number of `BitLinear` layers increased.

The BitBLAS library supported GEMM operations with $M > 1$, reducing the MLP's kernel calls per batch from 2,048 to 4. For the ViT, a chunked inference approach was still necessary because a kernel could not be compiled for the large $M = 25,216$ value. This approach resulted in a total of 194,040 kernel calls throughout inference, which was a significant improvement over the 48 million calls required using the GEMV kernel. The swap from a GEMV kernel to a GEMM kernel reduced the ViT's slowdown from 335x to 3.9x, demonstrating that GEMM kernels are more suitable for classification inference than GEMV kernels.

The memory savings from 1.58-bit quantization varied significantly across architectures. The MLP achieved a 15.8x reduced checkpoint size because all of its weights were contained in the quantizable `BitLinear` layers. By contrast, the ResNet achieved a negligible reduction in checkpoint size (from 0.72 MB to 0.7 MB) because its weights were dominated by convolutional layers that were incompatible with `BitLinear` layer replacement. The ViT achieved an intermediate 10.2x reduction due to its mix of quantizable layers and unquantizable components. Our results suggest that the memory savings of 1.58-bit quantization scale with the proportion of `BitLinear` layers in the architecture.

No inference speedups were observed in any architecture or kernel. Ma et al. explain that the inference speedups arise in part because the smaller weights generated from 1.58-bit quantization transfer more quickly from GPU HBM to the faster on-chip SRAM [6]. However, the dimensions of our architectures were small enough that memory transfers were not an issue from the start. Our ViT was the most promising candidate for demonstrating inference speedups, but was ultimately limited by the lack of a compatible GEMM kernel with support for large M values.

Chapter 7: Limitations

Several limitations of our methodology are worth acknowledging. First, our `BitLinear` layer implementation for the ViT experiments omitted RMSNorm, which is present in the original BitNet b1.58 implementation. Initially, the ViT was only achieving 1% accuracy during inference. Before the cause was narrowed down to kernel tiling incompatibility, we suspected RMSNorm may have been the cause because the ViT already has LayerNorm in each transformer block. We therefore removed RMSNorm from the `BitLinear` layer, but forgot to replace it once the true cause of the poor inference performance was determined. The omission of RMSNorm from the BitLinear layers in the ViT experiments may have caused the 5% accuracy gap observed between the Baseline and BitLinear models.

Before the kernel tiling constraints were fully understood, the MLP input was padded from 784 to 800 neurons during the custom W2A8 kernel experiments. Based on the kernel's internal tile size of 128, the correct padding should be 896, the next nearest multiple of 128 above 784. The improper padding means that the kernel skips the last 32 input features, 16 of which are non-zero. This should have produced incorrect inference results, but the model achieved 98.07% test accuracy. This is possibly because the skipped input features only comprise 2% ($16/784 \approx 2\%$) of the input, or because the MNIST dataset is simple enough that missing 2% of the pixels does not significantly alter classification results.

All experiments were performed in the Google Colab environment where GPU resources are shared between users. This may have contributed to the high variance observed in some inference speed metrics, making comparison less reliable than if the experiments were run on a local GPU.

GPU warmup was also only conducted for 5 batches of data before inference was performed. A longer warmup period may have led to more consistent inference speed measurements across runs. Additionally, each experiment was performed with only three runs. A larger number of runs would provide a more accurate measure of the standard deviation for applicable metrics.

Chapter 8: Conclusion

Over the course of this thesis, we applied 1.58-bit quantization to an MLP, a CNN, and a ViT with the goal of demonstrating accuracy preservation, checkpoint size reduction, and inference time speedups. We evaluated inference performance using both BitNet's custom W2A8 GEMV kernel and the GEMM kernels of the BitBLAS library.

Our experiments suggest accuracy preservation within 5% across all three architectures, supporting the claims of prior literature. Additionally, model checkpoint sizes were reduced by up to 15.8×, scaling with the percentage of layers compatible with `BitLinear` layer replacement. However, no inference speedups were observed using BitNet's custom GEMV kernel, as it was designed specifically for efficient inference during the LLM decode phase. We attempted to circumvent the kernel's $M = 1$ constraint by making additional kernel calls, but the additional kernel launch overhead increasingly slowed inference proportional to the number of `BitLinear` layers. Experiments with the GEMM kernels of the BitBLAS library dramatically improved inference speed results over the GEMV kernel, though performance did not improve over the cuBLAS baselines. Our results indicate that our model sizes were too small to stress GPU memory bandwidth, preventing the theoretical speedups from materializing. While we did not observe inference speedups in our testing, this result partially stems from the limitations of current low-bit GEMM kernels rather than a fundamental shortcoming of 1.58-bit quantization. With adequate low-bit kernel support, there is still potential for inference speedups, especially at larger model scales.

Overall, our findings suggest that 1.58-bit quantization is a promising approach to reducing checkpoint size without significant accuracy loss, particularly for architectures with a high proportion of layers suitable for `BitLinear` layer replacement. Since vision

models are increasingly deployed on resource-constrained edge devices where memory capacity and energy budgets are severely limited, reducing the memory footprint of these models through 1.58-bit quantization could enable more capable vision systems on devices such as smartphones, drones, and wearables.

Finally, our experiments highlight several potential avenues for future research. These include further developing existing low-bit GEMM kernels to support a wider range of matrix dimensions, testing larger model scales where inference speed is bottlenecked by GPU memory bandwidth, and evaluating 1.58-bit quantization on resource-constrained edge hardware. Pursuing these avenues of research will further clarify the advantages and tradeoffs of 1.58-bit quantization across a broader range of hardware scenarios.

References

[1] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, "Bitnet: Scaling 1-bit transformers for large language models," 2023. [Online]. Available: https://arxiv.org/abs/2310.11453

[2] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The era of 1-bit llms: All large language models are in 1.58 bits," 2024. [Online]. Available: https://arxiv.org/abs/2402.17764

[3] R. Gong, Y. Ding, Z. Wang, C. Lv, X. Zheng, J. Du, H. Qin, J. Guo, M. Magno, and X. Liu, "A survey of low-bit large language models: Basics, systems, and algorithms," 2025. [Online]. Available: https://arxiv.org/abs/2409.16694

[4] NVIDIA Corporation, "Cuda programming guide," 2024, accessed: 2026. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/index.html

[5] ——, "Matrix multiplication background user's guide," 2024, accessed: 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html

[6] S. Ma, H. Wang, S. Huang, X. Zhang, Y. Hu, T. Song, Y. Xia, and F. Wei, "Bitnet b1.58 2b4t technical report," 2025. [Online]. Available: https://arxiv.org/abs/2504.12285

[7] Z. Yuan, R. Zhou, H. Wang, L. He, Y. Ye, and L. Sun, "Vit-1.58b: Mobile vision transformers in the 1-bit era," 2024. [Online]. Available: https://arxiv.org/abs/2406.18051

[8] J. Nielsen and P. Schneider-Kamp, "Bitnet b1.58 reloaded: State-of-the-art performance also on smaller networks," 2024. [Online]. Available: https://arxiv.org/abs/2407.09527

[9] J. Nielsen, L. Galke, and P. Schneider-Kamp, "When are 1.58 bits enough? a bottom-up exploration of bitnet quantization," 2024. [Online]. Available: https://arxiv.org/abs/2411.05882

[10] X. Zuo, "Technical report: Multilayer perceptron on MNIST dataset," Machine Learning Lab, University of Freiburg, Technical Report, 2016.

[11] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, H.-L. Ru, F. Xue, S. Jiao, H. Wang, Z. Cheng, W. Li, and F. Wei, "The era of 1-bit LLMs: Training

tips, code and FAQ," Microsoft Research, Tech. Rep., 2024, gitHub repository. [Online]. Available: https://github.com/microsoft/unilm/blob/master/bitnet/The-Era-of-1-bit-LLMs___Training_Tips_Code_FAQ.pdf

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[13] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[14] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit, and L. Beyer, "How to train your vit? data, augmentation, and regularization in vision transformers," 2022. [Online]. Available: https://arxiv.org/abs/2106.10270

[15] L. Wang, L. Ma, S. Cao, Q. Zhang, J. Xue, Y. Shi, N. Zheng, Z. Miao, F. Yang, T. Cao, Y. Yang, and M. Yang, "Ladder: Enabling efficient low-precision deep learning computing through hardware-aware tensor transformation," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 307–323. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/wang-lei

Biography

Michael Tran is a Master's student in the Department of Computer Science at George Mason University, where he also completed his Bachelor's Degree. He initially studied on a pre-medical track at the University of Virginia, but later transitioned to Computer Science at GMU. His research interests lie in deep learning, with a particular focus on LLMs. He lives in Fairfax, Virginia, and spends his free time building and painting Gundam plastic models (Gunpla).